

Database Module Documentation

Business-style build record for PostgreSQL, Prisma schema generation, migration, seed data, troubleshooting, and validation outputs.

TIME BLOCK

10:45 AM - 11:45 AM

MODULE

Database

STATUS

Complete

Objective

Configure the project database layer so the platform has a PostgreSQL database, a Prisma data model, generated Prisma Client, applied schema migration, and demo seed data for the next modules.

Why This Module Matters

Persistent Data

Contacts, templates, campaigns, events, suppressions, users, and organisations all require a relational database.

Prisma Access

Prisma Client gives the API and worker type-safe database access for future modules.

Demo Readiness

Seed data gives the app a super admin, contacts, templates, segment, and campaign draft for testing.

Database Stack

Item	Selected Setup	Reason
Database	PostgreSQL 18.3	Required by the FRD and reliable for relational campaign data.
Port	5433	Your local PostgreSQL installation was running on this port.

ORM	Prisma 6.19.3	Type-safe schema, migration, client generation, and Studio UI.
Database Name	email_campaign_platform	Matches the platform purpose and environment configuration.
Seed Runner	tsx + dotenv-cli	Runs TypeScript seed files while loading root environment variables.

What We Did

Step	Action	Why
1	Installed PostgreSQL manually.	The machine did not have PostgreSQL installed, and the user chose manual installation instead of Docker.
2	Skipped Stack Builder.	Stack Builder is optional and not required for this app.
3	Created/used database email_campaign_platform.	Prisma needs an existing PostgreSQL database target for migration.
4	Updated DATABASE_URL to port 5433.	Your PostgreSQL server was configured on 5433 rather than the default 5432.
5	Installed Prisma packages at compatible version 6.19.3.	Prisma CLI attempted to use 6.19.3, so the package versions were aligned.
6	Updated API Prisma scripts to use dotenv-cli.	The Prisma commands run from apps/api and need root .env values.
7	Removed conflicting prisma/.env usage.	Duplicate DATABASE_URL values caused Prisma Studio conflict.
8	Generated Prisma Client.	Required before using Prisma Client in API and seed code.
9	Ran migration.	Created all database tables from prisma/schema.prisma.
10	Ran seed script.	Inserted demo organisation, admin user, contacts, templates, segment, and campaign draft.
11	Ran typecheck and build.	Confirmed the database changes did not break the monorepo.

Environment Configuration

The final root environment file uses PostgreSQL on localhost port 5433.

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5433/email_campaign_platform?schema=public"
JWT_ACCESS_SECRET="change-me-access-secret"
JWT_REFRESH_SECRET="change-me-refresh-secret"
ACCESS_TOKEN_MINUTES="15"
REFRESH_TOKEN_DAYS="7"
API_PORT="4000"
WEB_ORIGIN="http://localhost:5173"
AWS_REGION="ap-south-1"
AWS_ACCESS_KEY_ID=""
AWS_SECRET_ACCESS_KEY=""
SES_CONFIGURATION_SET="email-platform-events"
SES_FROM_EMAIL="verified@example.com"
S3_BUCKET=""
```

Note: This is local development configuration. Real secrets should not be committed to GitHub.

Prisma Scripts

The API package scripts were updated so Prisma always loads the root .env file.

```
"prisma:generate": "dotenv -e ../../.env -- prisma generate --schema ../../prisma/schema.prisma",
"prisma:migrate": "dotenv -e ../../.env -- prisma migrate dev --schema ../../prisma/schema.prisma",
"prisma:seed": "dotenv -e ../../.env -- tsx src/seed.ts",
"prisma:studio": "dotenv -e ../../.env -- prisma studio --schema ../../prisma/schema.prisma"
```

Commands Run

```
pnpm add -w -D prisma@6.19.3
pnpm add -w @prisma/client@6.19.3
pnpm --filter api add -D prisma@6.19.3
pnpm --filter api add @prisma/client@6.19.3
pnpm install
pnpm db:generate
pnpm db:migrate
pnpm db:seed
pnpm typecheck
pnpm build
```

Successful Outputs

Prisma Generate

```
Prisma schema loaded from ../../prisma/schema.prisma
Generated Prisma Client (v6.19.3)
```

Prisma Migrate

```
Datasource "db": PostgreSQL database "email_campaign_platform", schema "public" at "localhost:5433"
Already in sync, no schema change or pending migration was found.
Generated Prisma Client (v6.19.3)
```

Prisma Seed

```
Database seeded successfully.
Admin login:
Email: admin@example.com
Password: Admin@123
```

Root Typecheck

```
apps/api typecheck completed
apps/web typecheck completed
apps/worker typecheck completed
```

Root Build

```
apps/web build completed
apps/api build completed
apps/worker build completed
```

Seed Data Created

Model	Expected Count	Purpose
Organisation	1	Tenant container for users, contacts, templates, campaigns, and settings.
User	1	Super admin login for future auth module.
ContactList	1	Newsletter Subscribers demo list.
Contact	3	Sample recipients for campaign testing.
ContactListMember	3	Links all three contacts to the demo list.
Template	5	Required FRD starter templates.

Segment	1	Sample smart segment for future segmentation module.
Campaign	1	Draft campaign for future campaign workflow testing.

Admin Login

Field	Value
Email	admin@example.com
Password	Admin@123
Role	SUPER_ADMIN

Issues Found and Fixed

Prisma auto-install failed: Prisma attempted to run `pnpm add prisma@6.19.3` automatically. We fixed this by installing Prisma and Prisma Client manually.

DATABASE_URL not found: Prisma commands run from `apps/api` could not see the root `.env`. We fixed this with `dotenv-cli` in Prisma scripts.

Prisma Studio env conflict: Both root `.env` and `prisma/.env` had `DATABASE_URL`. We consolidated to the root `.env`.

Authentication failed: PostgreSQL credentials needed to match the actual local server and port. The final working URL uses `localhost:5433`.

Empty database in Studio: Tables existed but counts were zero before seed ran successfully. The seed script populated the required demo data.

Completion Status

Database Module Complete

The database is migrated, Prisma Client is generated, seed data exists, and the full monorepo passes typecheck and build.

Next Module

AWS Base Setup: add AWS environment variables, AWS SDK packages, SES/dev mailer wrapper, S3 upload helper, SQS queue helper, and a temporary dev email test route.

Code Appendix

The following appendix contains the relevant database module files from the repo.

apps/api/package.json

```
{
  "name": "api",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "main": "dist/index.js",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc -p tsconfig.json",
    "start": "node dist/index.js",
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "lint": "echo \"lint api\"",
    "prisma:generate": "dotenv -e ../../.env -- prisma generate --schema ../../prisma/schema.prisma",
    "prisma:migrate": "dotenv -e ../../.env -- prisma migrate dev --schema
../../prisma/schema.prisma",
    "prisma:seed": "dotenv -e ../../.env -- tsx src/seed.ts",
    "prisma:studio": "dotenv -e ../../.env -- prisma studio --schema ../../prisma/schema.prisma"
  },
  "dependencies": {
    "@prisma/client": "^6.19.3",
    "bcryptjs": "^3.0.2",
    "cors": "^2.8.5",
    "dotenv": "^16.5.0",
    "express": "^5.1.0",
    "jsonwebtoken": "^9.0.2",
    "zod": "^3.24.4"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.6",
    "@types/cors": "^2.8.17",
    "@types/express": "^5.0.1",
    "@types/jsonwebtoken": "^9.0.9",
    "@types/node": "^22.15.17",
    "dotenv-cli": "^7.4.4",
    "prisma": "^6.19.3",
    "tsx": "^4.19.4",
    "typescript": "^5.8.3"
  }
}
```

prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
```

```
    url      = env("DATABASE_URL")
}

enum UserRole {
    SUPER_ADMIN
    CAMPAIGN_MANAGER
    VIEWER
}

enum ContactStatus {
    ACTIVE
    UNSUBSCRIBED
    BOUNCED
    COMPLAINED
}

enum ContactSource {
    IMPORT
    FORM
    MANUAL
    API
}

enum CampaignStatus {
    DRAFT
    SCHEDULED
    SENDING
    SENT
    PAUSED
    CANCELLED
}

enum SendStatus {
    QUEUED
    SENT
    DELIVERED
    BOUNCED
    COMPLAINED
    UNSUBSCRIBED
    FAILED
}

enum EventType {
    SENT
    DELIVERED
    OPENED
    CLICKED
    BOUNCED
    COMPLAINED
    UNSUBSCRIBED
}

enum SuppressionReason {
    BOUNCED
    COMPLAINED
    UNSUBSCRIBED
    MANUAL
}

enum ImportStatus {
```



```

    PENDING
    RUNNING
    COMPLETED
    FAILED
}

model Organisation {
  id          String    @id @default(cuid())
  name        String
  logoUrl     String?
  fromEmail   String?
  sesConfigSet String?
  awsRegion   String    @default("ap-south-1")
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  users       User[]
  lists       ContactList[]
  contacts    Contact[]
  segments    Segment[]
  templates   Template[]
  campaigns   Campaign[]
  suppressions SuppressionList[]
}

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String?
  passwordHash String
  role        UserRole  @default(CAMPAIGN_MANAGER)
  orgId       String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  organisation Organisation @relation(fields: [orgId], references: [id], onDelete: Cascade)
}

model ContactList {
  id          String    @id @default(cuid())
  orgId       String
  name        String
  description String?
  tags        String[]  @default([])
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  organisation Organisation @relation(fields: [orgId], references: [id], onDelete: Cascade)
  members      ContactListMember[]
  importJobs   ImportJob[]

  @@unique([orgId, name])
}

model Contact {
  id          String    @id @default(cuid())
  orgId       String
  email       String
  firstName   String?
  lastName    String?

```

```

phone      String?
status     ContactStatus @default(ACTIVE)
customFields Json       @default("{}")
source     ContactSource @default(MANUAL)
createdAt  DateTime      @default(now())
updatedAt  DateTime      @updatedAt

organisation Organisation @relation(fields: [orgId], references: [id], onDelete: Cascade)
lists      ContactListMember[]
sends      CampaignSend[]
events     EmailEvent[]

@@unique([orgId, email])
}

model ContactListMember {
  contactId String
  listId     String
  subscribedAt DateTime @default(now())

  contact Contact @relation(fields: [contactId], references: [id], onDelete: Cascade)
  list      ContactList @relation(fields: [listId], references: [id], onDelete: Cascade)

  @@id([contactId, listId])
}

model Segment {
  id      String @id @default(cuid())
  orgId   String
  name    String
  rules   Json
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  organisation Organisation @relation(fields: [orgId], references: [id], onDelete: Cascade)
}

model Template {
  id      String @id @default(cuid())
  orgId   String
  name    String
  category String
  blocks  Json @default("[]")
  html    String
  thumbnailUrl String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  organisation Organisation @relation(fields: [orgId], references: [id], onDelete: Cascade)
  campaigns Campaign[]
}

model Campaign {
  id      String @id @default(cuid())
  orgId   String
  name    String
  subject String
  previewText String?
  fromName String
  fromEmail String

```

```

replyToEmail String?
status      CampaignStatus @default(DRAFT)
templateId  String?
scheduledAt DateTime?
timezone    String?
createdAt   DateTime        @default(now())
updatedAt   DateTime        @updatedAt

organisation Organisation @relation(fields: [orgId], references: [id], onDelete: Cascade)
template    Template?    @relation(fields: [templateId], references: [id])
sends       CampaignSend[]
events      EmailEvent[]
}

model CampaignSend {
  id          String      @id @default(cuid())
  campaignId  String
  contactId   String
  sesMessageId String?
  status      SendStatus @default(QUEUED)
  sentAt      DateTime?
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  campaign Campaign @relation(fields: [campaignId], references: [id], onDelete: Cascade)
  contact  Contact  @relation(fields: [contactId], references: [id], onDelete: Cascade)

  @@unique([campaignId, contactId])
}

model EmailEvent {
  id          String      @id @default(cuid())
  contactId   String?
  campaignId  String?
  eventType   EventType
  occurredAt  DateTime    @default(now())
  ip          String?
  userAgent   String?
  metadata    Json        @default("{}")

  contact  Contact? @relation(fields: [contactId], references: [id], onDelete: SetNull)
  campaign Campaign? @relation(fields: [campaignId], references: [id], onDelete: SetNull)

  @@index([campaignId, eventType])
  @@index([contactId, eventType])
}

model SuppressionList {
  id          String      @id @default(cuid())
  orgId       String
  email       String
  reason      SuppressionReason
  suppressedAt DateTime    @default(now())

  organisation Organisation @relation(fields: [orgId], references: [id], onDelete: Cascade)

  @@unique([orgId, email])
}

model ImportJob {

```

```

id      String      @id @default(cuid())
listId   String
status   ImportStatus @default(PENDING)
totalRows Int        @default(0)
added    Int          @default(0)
updated  Int          @default(0)
skipped  Int          @default(0)
errored  Int          @default(0)
startedAt DateTime?
endedAt   DateTime?
createdAt DateTime    @default(now())

list ContactList @relation(fields: [listId], references: [id], onDelete: Cascade)
}

```

apps/api/src/seed.ts

```

import "dotenv/config";
import { PrismaClient, UserRole } from "@prisma/client";
import bcrypt from "bcryptjs";

const prisma = new PrismaClient();

const starterTemplates = [
  {
    name: "Welcome",
    category: "Onboarding",
    html: `
      <div style="font-family:Arial,sans-serif;max-width:600px;margin:0 auto;padding:24px;">
        <h1 style="color:#172033;">Welcome {{first_name}}</h1>
        <p>Thanks for joining us. We are glad to have you here.</p>
        <a href="https://example.com" style="display:inline-block;background:#1f5eff;color:#fff;padding:12px 18px;text-decoration:none;border-radius:6px;">Get
        Started</a>
        <p style="font-size:12px;color:#687386;margin-top:32px;">{{unsubscribe_link}}</p>
      </div>
    `,
  },
  {
    name: "Newsletter",
    category: "Newsletter",
    html: `
      <div style="font-family:Arial,sans-serif;max-width:600px;margin:0 auto;padding:24px;">
        <h1 style="color:#172033;">Monthly Newsletter</h1>
        <p>Hello {{first_name}}, here are this month's updates.</p>
        <hr style="border:none;border-top:1px solid #d8dee9;" />
        <p>Feature update, product news, and useful resources.</p>
        <p style="font-size:12px;color:#687386;margin-top:32px;">{{unsubscribe_link}}</p>
      </div>
    `,
  },
  {
    name: "Promotional",
    category: "Marketing",
    html: `
      <div style="font-family:Arial,sans-serif;max-width:600px;margin:0 auto;padding:24px;">
        <h1 style="color:#172033;">Special Offer</h1>
        <p>Hi {{first_name}}, this offer is available for a limited time.</p>
      </div>
    `,
  },
];

```



```

    },
    create: {
      email: "admin@example.com",
      name: "Super Admin",
      passwordHash,
      role: UserRole.SUPER_ADMIN,
      orgId: organisation.id
    }
  });

const list = await prisma.contactList.upsert({
  where: {
    orgId_name: {
      orgId: organisation.id,
      name: "Newsletter Subscribers"
    }
  },
  update: {},
  create: {
    orgId: organisation.id,
    name: "Newsletter Subscribers",
    description: "Seed list for demo contacts",
    tags: ["demo", "newsletter"]
  }
});

const contacts = [
  {
    email: "aisha@example.com",
    firstName: "Aisha",
    lastName: "Rao",
    customFields: {
      company: "Acme",
      city: "Chennai"
    }
  },
  {
    email: "rahul@example.com",
    firstName: "Rahul",
    lastName: "Mehta",
    customFields: {
      company: "Northstar",
      city: "Bengaluru"
    }
  },
  {
    email: "priya@example.com",
    firstName: "Priya",
    lastName: "Sharma",
    customFields: {
      company: "Bluepeak",
      city: "Mumbai"
    }
  }
];

for (const item of contacts) {
  const contact = await prisma.contact.upsert({
    where: {
      orgId_email: {

```

```

        orgId: organisation.id,
        email: item.email
    }
},
update: {
    firstName: item.firstName,
    lastName: item.lastName,
    customFields: item.customFields
},
create: {
    orgId: organisation.id,
    email: item.email,
    firstName: item.firstName,
    lastName: item.lastName,
    source: "MANUAL",
    customFields: item.customFields
}
});

await prisma.contactListMember.upsert({
    where: {
        contactId_listId: {
            contactId: contact.id,
            listId: list.id
        }
    },
    update: {},
    create: {
        contactId: contact.id,
        listId: list.id
    }
});
}

const createdTemplates = [];

for (const template of starterTemplates) {
    const created = await prisma.template.upsert({
        where: {
            id: `seed-template-${template.name.toLowerCase().replaceAll(" ", "-")}`
        },
        update: {
            name: template.name,
            category: template.category,
            html: template.html,
            blocks: []
        },
        create: {
            id: `seed-template-${template.name.toLowerCase().replaceAll(" ", "-")}`,
            orgId: organisation.id,
            name: template.name,
            category: template.category,
            html: template.html,
            blocks: []
        }
    });

    createdTemplates.push(created);
}

```

```

await prisma.segment.upsert({
  where: {
    id: "seed-segment-chennai"
  },
  update: {},
  create: {
    id: "seed-segment-chennai",
    orgId: organisation.id,
    name: "City equals Chennai",
    rules: {
      operator: "AND",
      conditions: [
        {
          field: "custom.city",
          operator: "equals",
          value: "Chennai"
        }
      ]
    }
  }
});

```

```

await prisma.campaign.upsert({
  where: {
    id: "seed-campaign-draft"
  },
  update: {},
  create: {
    id: "seed-campaign-draft",
    orgId: organisation.id,
    name: "Demo Welcome Campaign",
    subject: "Welcome to Demo Organisation",
    previewText: "A quick welcome message for new subscribers",
    fromName: "Demo Marketing",
    fromEmail: "verified@example.com",
    replyToEmail: "verified@example.com",
    templateId: createdTemplates[0]?.id,
    status: "DRAFT",
    timezone: "Asia/Kolkata"
  }
});

```

```

console.log("Database seeded successfully.");
console.log("Admin login:");
console.log("Email: admin@example.com");
console.log("Password: Admin@123");
}

```

```

main()
  .catch((error) => {
    console.error(error);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });

```